EQUIPO:
Melvin Adrian Rios López
Hugo Ivan Aguilar Barron
Paulina Mane Davalos
No nos repruebe profe:(

Modelo de Aula Invertida

Tema: Ordenamiento Básico (algoritmos cuadráticos) **Evidencia:** Reporte comparativo de algoritmos de ordenamiento básico

La semana se estructura así dentro del marco AETL:

COMPRENDE — Mecánica visual de los tres algoritmos: Bubble Sort, Selection Sort e Insertion Sort. Trazado paso a paso sobre un arreglo de ejemplo. Identificación de invariantes de ciclo en cada uno. Análisis de complejidad $O(n^2)$ en peor caso, $O(n)$ en mejor caso para Insertion Sort, y noción de estabilidad.

APLICA — Implementación de los tres algoritmos desde cero en el lenguaje del curso. Un único reto de dos etapas: primero implementar cada algoritmo con sus comentarios # DECISIÓN:, luego ejecutar benchmarking comparativo con arreglos de tamaños crecientes (n = 100, 1 000, 10 000) y distintas distribuciones (ordenado, inverso, aleatorio).

REFLEXIONA — Defensa verbal de al menos una decisión de diseño: ¿por qué Insertion Sort es O(n) en mejor caso? ¿Cuándo elegirías Selection Sort sobre Bubble Sort? Análisis de qué predicó la teoría vs. lo que arrojaron los benchmarks.

VALIDA — Checklist de casos de prueba: arreglo vacío, un elemento, ya ordenado, inversamente ordenado, con duplicados. Verificación de que los tiempos empíricos son consistentes con las complejidades teóricas.

PROFUNDIZA — Conexión anticipatoria hacia Semana 9: ¿qué limitación concreta de estos algoritmos motiva el paradigma divide y vencerás?

COMPRENDE

Bubble sort: Es el algoritmo de clasificación más simple que funciona intercambiando los elementos o datos adyacentes si están en el orden incorrecto, no es muy eficiente para grandes cantidades de datos pues su complejidad de tiempo es de O(n^2)

Invariante de ciclo: al terminar la i-esima iteración, los últimos i elementos están en su posición definitiva

Complejidad:

- Peor caso: O(n^2)
- Mejor caso: O(n^2)

Estabilidad: Si, conserva el orden relativo de elementos iguales

Código

```
#include <bits/stdc++.h>
using namespace std;
void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    bool swapped;

    for (int i = 0; i < n - 1; i++) {
        swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }

        if (!swapped)
            break;
    }
}

void printVector(const vector<int>& arr) {
    for (int num : arr)
        cout << " " << num;
}

int main() {
    vector<int> arr = { 64, 34, 25, 12, 22, 11, 90 };
    bubbleSort(arr);
    cout << "Sorted array: \n";
    printVector(arr);
    return 0;
}
```

Resultado final

```
Sorted array:
 11 12 22 25 34 64 90
```

Selection Sort: Este algoritmo funciona seleccionando repetidamente el dato mas pequeño (o el mas grande) de la parte sin clasificar e intercambiandolo con el primer elemento sin clasificar

Invariante de ciclo: Al inicio de cada iteración, la sublista izquierda ya está ordenada y contiene los elementos más pequeños

Complejidad:

- Peor caso:O(n^2)
- Mejor caso: O(n^2)

Estabilidad: No es estable, por que al intercambiar puede alterar el orden relativo de duplicados

Código

```
#include <bits/stdc++.h>
using namespace std;

void selectionSort(vector<int> &arr) {
    int n = arr.size();

    for (int i = 0; i < n - 1; ++i) {

        int min_idx = i;

        for (int j = i + 1; j < n; ++j) {
            if (arr[j] < arr[min_idx]) {

                min_idx = j;
            }
        }

        swap(arr[i], arr[min_idx]);
    }
}

void printArray(vector<int> &arr) {
    for (int &val : arr) {
        cout << val << " ";
    }
    cout << endl;
}

int main() {
    vector<int> arr = {64, 25, 12, 22, 11};

    cout << "Original array: ";
    printArray(arr);

    selectionSort(arr);

    cout << "Sorted array: ";
    printArray(arr);

    return 0;
}
```

Resultado Final

```
Original array: 64 25 12 22 11
Sorted array: 11 12 22 25 64
```

Insertion sort: Es un algoritmo de clasificación simple que funciona insertando iterativamente cada elemento de la lista sin clasificar en su posición en una parte ordenada de la lista

Invariante de ciclo: La sublista izquierda siempre esta ordenada despues de cada insercion

Complejidad:

- Peor caso:O(n^2)
- Mejor caso:O(n)

Estabilidad: Si tiene estabilidad, mantiene el orden relativo de duplicados

Código

```
#include <iostream>
using namespace std;

void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; ++i) {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}


void printArray(int arr[], int n)
{    for (int i = 0; i < n; ++i)
        cout << arr[i] << " ";
    cout << endl;
}

int main()
{
    int arr[] = { 12, 11, 13, 5, 6 };
    int n = sizeof(arr) / sizeof(arr[0]);

    insertionSort(arr, n);
    printArray(arr, n);

    return 0;
}
```

Resultado final

```
5 6 11 12 13
```

APLICA

Código

```
#include <iostream>
#include <vector>
#include <chrono> // Necesaria para medir el tiempo en milisegundos con precisión
#include <cstdlib> // Para rand() y srand()

using namespace std;
using namespace std::chrono;

// ==========================================
// 1. IMPLEMENTACIÓN DE LOS ALGORITMOS
// ==========================================

void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    bool intercambiado;
    for (int i = 0; i < n - 1; i++) {
```

```
        intercambiado = false;
        // DECISIÓN: Se implementa una bandera 'intercambiado' (swapped).
        // Si en una pasada completa no hubo intercambios, el arreglo ya está
        // ordenado y rompemos el ciclo. Esto mejora el mejor caso a O(n).
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                intercambiado = true;
            }
        }
        if (!intercambiado) break;
    }
}

void selectionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        int indice_minimo = i;
        // DECISIÓN: Solo hacemos el intercambio de variables al final del ciclo interno,
        // después de haber escaneado todo el resto del arreglo. Esto garantiza un máximo
        // de O(n) intercambios totales en memoria, lo cual es la principal ventaja de este algoritmo.
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[indice_minimo]) {
                indice_minimo = j;
            }
        }
        if (indice_minimo != i) {
            int temp = arr[i];
            arr[i] = arr[indice_minimo];
            arr[indice_minimo] = temp;
        }
    }
}

void insertionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 1; i < n; i++) {
        int clave = arr[i];
        int j = i - 1;
        // DECISIÓN: En lugar de intercambiar elementos iterativamente (lo que requiere 3 asignaciones
        // por paso), guardamos el elemento a insertar en la variable 'clave' y simplemente
        // desplazamos los elementos mayores hacia la derecha. Esto reduce enormemente las asignaciones.
        while (j >= 0 && arr[j] > clave) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = clave;
    }
}

// ==========================================
// 2. BENCHMARKING COMPARATIVO
// ==========================================

// Función auxiliar para copiar el arreglo, ordenarlo y medir el tiempo
double medirTiempo(void (*funcionOrdenamiento)(vector<int>&), const vector<int>& arregloOriginal) {
    vector<int> copiaArreglo = arregloOriginal; // Trabajamos con una copia para no alterar el original

    auto inicio = high_resolution_clock::now();
    funcionOrdenamiento(copiaArreglo);
    auto fin = high_resolution_clock::now();

    duration<double, std::milli> tiempo_ms = fin - inicio;
    return tiempo_ms.count();
```

```
}

void ejecutarPruebas(int n) {
    cout << "\n--- Resultados para N = " << n << " ---" << endl;

    vector<int> ordenado(n), inverso(n), aleatorio(n);

    // Poblar los arreglos según la distribución
    for(int i = 0; i < n; i++) {
        ordenado[i] = i;
        inverso[i] = n - i;
        aleatorio[i] = rand() % 10000; // Números aleatorios simples
    }

    cout << "Distribucion\tBubble (ms)\tSelection (ms)\tInsertion (ms)" << endl;

    // Prueba Ordenado
    cout << "Ordenado\t"
         << medirTiempo(bubbleSort, ordenado) << "\t\t"
         << medirTiempo(selectionSort, ordenado) << "\t\t"
         << medirTiempo(insertionSort, ordenado) << endl;

    // Prueba Inverso
    cout << "Inverso \t"
         << medirTiempo(bubbleSort, inverso) << "\t\t"
         << medirTiempo(selectionSort, inverso) << "\t\t"
         << medirTiempo(insertionSort, inverso) << endl;

    // Prueba Aleatorio
    cout << "Aleatorio\t"
         << medirTiempo(bubbleSort, aleatorio) << "\t\t"
         << medirTiempo(selectionSort, aleatorio) << "\t\t"
         << medirTiempo(insertionSort, aleatorio) << endl;
}

int main() {
    srand(time(0)); // Inicializar la semilla para los números aleatorios

    // Ejecutar para los distintos tamaños solicitados
    int tamanos[] = {100, 1000, 10000};

    for (int n : tamanos) {
        ejecutarPruebas(n);
    }

    return 0;
}
```

Resultado final

```
--- Resultados para N = 100 ---
Distribucion    Bubble (ms) Selection (ms)  Insertion (ms)
Ordenado    0.00115     0.04474     0.0015
Inverso     0.11421     0.04561     0.06095
Aleatorio   0.08037     0.04809     0.02886

--- Resultados para N = 1000 ---
Distribucion    Bubble (ms) Selection (ms)  Insertion (ms)
Ordenado    0.00754     2.84926     0.00639
Inverso     5.7548      2.15553     2.88357
Aleatorio   4.03881     2.29568     1.4358

--- Resultados para N = 10000 ---
Distribucion    Bubble (ms) Selection (ms)  Insertion (ms)
Ordenado    0.04013     244.526     0.06198
Inverso     563.438     224.216     290.108
Aleatorio   396.379     226.976     147.355
```

REFLEXIONA

1. Defensa:Insertion Sort en tiempo O(n):

Insertion Sort es eficiente en su mejor caso porque sabe cuándo parar rápido.Otros programas terminan sus tareas sin pensar, pero este programa de inserción solo funciona si lo que está a la izquierda es más grande que lo que se necesita.Si el arreglo ya está ordenado, la condición es falsa desde el primer intento. Así, se eliminan los desplazamientos. De esta manera, solo se hacen n comparaciones y ningún movimiento, lo que hace que su complejidad sea lineal.

2. Selección: Selection vs. Bubble Sort:

Elegiría ordenar por selección en lugar de ordenar por burbuja cuando el costo de escritura en la memoria sea muy importante, por ejemplo, en sistemas integrados o tarjetas SD que tienen un ciclo de vida limitado.Bubble Sort a veces necesita hacer muchos movimientos, hasta n al cuadrado intercambios, mientras que Selection Sort es diferente. Este siempre hace como máximo n intercambios, uno por cada lugar que necesita ordenar. Por eso, es mucho más amable con el hardware de la computadora.

3. Teoría vs. Benchmarks:

Los resultados de las pruebas confirman lo que pensábamos: cuando N pasó de 1,000 a 10,000, el tiempo se multiplicó por cien, lo que demuestra un crecimiento de tipo O(n^2).Sensibilidad al Orden: Insert Sort fue el más rápido cuando los arreglos ya estaban ordenados."Lo que pasa con Selection Sort es que siempre dura casi lo mismo, no importa cómo estén los datos de ordenados o desordenados al principio. Es que siempre busca el número más pequeño de la misma manera, revisando todo."

VÁLIDA

Caso de prueba	Bubble sort	Selection sort	Insertion sort	Resultado esperado
Arreglo vacío	☑	☑	☑	Devuelve arreglo vacío
Un solo elemento	☑	☑	☑	Devuelve el mismo elemento
Arreglo ya ordenado	☑	☑	☑	Mantiene orden, Insertion Sort ≈ O(n)
Arreglo inversamente ordenado	☑	☑	☑	Ordena correctamente, todos ≈ O(n²)
Arreglo con duplicados	☑	☑	☑	Ordena correctamente, estabilidad en Bubble e Insertion

Profundiza

Lo que nos hace cambiar un poco el chip es que, con tantos datos, el tiempo se nos va de las manos bien rápido.Con los algoritmos cuadráticos que usaste, como Bubble, Selection e Insertion, el trabajo que se hace va de a poquito: para meter un solo elemento en el grupo que ya está ordenado, tenemos que compararlo con casi todos los demás.

Así que, esto crea un problema importante: si hacemos el arreglo el doble de grande, el programa no tardará el doble, sino cuatro veces más.¿Por qué se usa tanto eso de "Divide y Vencerás"?Algoritmos como Merge Sort o Quick Sort. Lo que hacen es buscar menos: dividen el arreglo a la mitad una y otra vez. Así, cada parte tiene que compararse muy pocas veces, en un número logarítmico.Ahora, el costo que antes era mucho más alto, baja significativamente con esta mejora.

"Piensa en esto: si tienes que ordenar un millón de cosas, un método que es como 'n al cuadrado' tendría que hacer un billón de pasos. Pero si usas la táctica de 'divide y vencerás', solo harías unos 20 millones. La diferencia es enorme.

"Esa diferencia hace que las aplicaciones de hoy puedan manejar muchísimos datos en un instante.